

Relatório Técnico: Sistema de Matchmaking para Jogos Online

Trabalho 3 – Estruturas de Dados
Gabriel Bittencourt Dias e Lucas Gabriel de Oliveira

17 de maio de 2026

1 Implementação dos Algoritmos de Ordenação

O sistema requer a organização dos jogadores na fila de espera com base no nível de habilidade (*score*) e, em caso de empate, na ordem de chegada (*timestamp*). Para isso, foram implementados manualmente dois algoritmos de ordenação, respeitando a restrição de não utilizar funções prontas da STL.

- **Insertion Sort:** A implementação segue uma abordagem iterativa. O algoritmo percorre o array a partir do segundo elemento, comparando-o com os anteriores. Caso o elemento atual possua um *score* menor (ou *score* igual e *timestamp* menor), os elementos maiores são deslocados uma posição para a direita para abrir espaço, inserindo o jogador na sua posição correta. É um método *in-place*, simples, mas ineficiente para grandes volumes de dados.
- **Merge Sort:** Implementado utilizando o paradigma de divisão e conquista com o uso de recursão. O algoritmo divide o array de jogadores sucessivamente pela metade até restarem sub-arrays de tamanho 1. Em seguida, a função `merge` é chamada recursivamente para combinar essas metades. Durante a intercalação (*merge*), um array auxiliar é alocado dinamicamente para comparar os elementos das duas metades (sempre avaliando o *score* e o *timestamp*) e copiá-los de volta para o array original de forma ordenada. Para evitar vazamento de memória, os arrays temporários são liberados a cada retorno da recursão.

2 Formação de Grupos via Ordenação

A estratégia de ordenar os jogadores previamente transforma um problema complexo de combinação em uma busca linear simples. Como o array de jogadores já se encontra em ordem crescente de *score*, jogadores com níveis de habilidade semelhantes ficam necessariamente em posições adjacentes.

Para formar um grupo, o algoritmo utiliza uma abordagem de “janela deslizante” (*sliding window*) de tamanho `groupSize`. Ele percorre o array ordenado verificando apenas se a diferença entre o último jogador da janela (maior *score*) e o primeiro (menor *score*) é menor ou igual ao `delta` permitido. Ao encontrar o primeiro grupo válido, os jogadores são alocados dinamicamente para o retorno e removidos do array original (deslocando os jogadores restantes para preencher a lacuna).

3 Análise de Custo Computacional

Abaixo está a análise de complexidade assintótica (Notação Big-O) das principais operações do sistema, considerando N como o número atual de jogadores na fila de espera:

- **Inserção (`insert`):** $O(1)$. Como os jogadores são sempre inseridos no final do array estático (controlado pela variável `size`), não há necessidade de deslocamento de elementos.

- **Remoção (removePlayer):** $O(N)$ no pior caso. É necessário buscar linearmente o jogador pelo ID e, ao encontrá-lo, deslocar todos os jogadores subsequentes uma posição para a esquerda para manter a contiguidade do array.
- **Ordenação (Insertion Sort):** $O(N^2)$ no pior caso e caso médio, pois para cada elemento inserido, pode ser necessário deslocar todos os elementos anteriores. É $O(N)$ apenas no melhor caso (array já ordenado).
- **Ordenação (Merge Sort):** $O(N \log N)$ em todos os casos. A divisão recursiva custa $\log N$, e a intercalação dos elementos custa N , garantindo um desempenho consistente.
- **Formação de Grupo (formGroup):** $O(N)$. A verificação da janela deslizante percorre o array ordenado em tempo linear. A remoção subsequente dos jogadores do grupo também tem custo linear, pois exige o deslocamento dos elementos restantes.
- **Recuperação (getWaitingPlayers):** $O(N)$. É necessário percorrer o array atual alocando dinamicamente e copiando cada um dos N jogadores iterativamente.

4 Comparação Experimental: Insertion Sort vs. Merge Sort

Para validar a eficiência teórica na prática, foram realizados testes de desempenho medindo o tempo de execução de ambos os algoritmos em C++.

Metodologia de Teste: Foram gerados arrays de jogadores com dados aleatórios de *score* e *timestamp*. O tempo de execução foi medido utilizando a biblioteca <chrono>, rodando ambos os algoritmos sob os mesmos conjuntos de dados de diferentes tamanhos (N).

Tabela 1: Tempos de execução observados para ordenação

Tamanho da Entrada (N)	Tempo - Insertion Sort	Tempo - Merge Sort
5.000 jogadores	0.3327 s	0.0130 s
10.000 jogadores	2.0222 s	0.0534 s
20.000 jogadores	6.0273 s	0.0722 s

Discussão dos Resultados:

Os resultados observados estão perfeitamente alinhados com a complexidade assintótica esperada. Para entradas pequenas, ambos os algoritmos resolvem o problema rapidamente. No entanto, conforme a entrada cresce, o custo $O(N^2)$ do Insertion Sort fica evidente. Observa-se que ao dobrar o volume de dados de 10.000 para 20.000, o tempo de execução do Insertion Sort sofre um aumento aproximadamente quadrático (saltando de 2.0222 s para 6.0273 s).

Em contrapartida, o Merge Sort prova a sua eficiência log-linear $O(N \log N)$, resolvendo o maior volume de dados em uma fração de segundo. Isso demonstra sua excelente escalabilidade, justificando-se como a escolha ideal e necessária para o cenário do projeto, que prevê suportar até 100.000 jogadores.